PLANO DE TESTE: A&F Health Management

André Coutinho de Macedo Silva
André Rodrigues Farias da Silva
Fernanda Maria de Souza

Centro Universitário Senac

PLANO DE TESTE

O objetivo deste documento é descrever o Plano de Teste do Projeto de Software A&F Health Management. Ele contém a definição do escopo do Projeto de Software, a especificação dos requisitos do sistema, a estratégia de teste e o esforço necessário para sua realização. Na estratégia de teste estão definidos as técnicas, níveis e tipos de teste que serão executados na iteração e os objetivos que devem ser atingidos. O documento apresenta ainda a definição do ambiente de testes, a matriz de responsabilidade e a relação dos casos de testes. Em seguida são listados os riscos de teste, os critérios para conclusão dos testes e o cronograma do projeto de teste.

Sumário

1 ESCOPO DO PRODUTO DE sOFTWARE 4
1.2 Objetivo do Projeto 4
1.3 Metodologia de desenvolvimento 5
2 Matriz de rastreabilidade de requisitos de sistema 6
2.1 Requisitos Funcionais 6
2.2. Requisitos Não Funcionais 6
3 ESTRATÉGIA DE TESTE 8
4 AMBIENTE de TESTE 8
5 MATRIZ DE RESPONSABILIDADE 10
6 RELAÇÃO e descrição DOS CASOS DE TESTES 13
6.1 Descrição dos principais cenários de testes 19
7 RISCOS DO PROJETO DE TESTE DE SOFTWARE 20
8 CRITÉRIOS DE CONCLUSÃO 22
9 CRONOGRAMA DO PROJETO DE TESTE DE SOFTWARE 23
REFERÊNCIAS 26

1 ESCOPO DO PRODUTO DE SOFTWARE

A declaração do escopo do produto de software descreve, em detalhes, as entregas do produto e o trabalho necessário para criar essas entregas. Fornece um entendimento comum do escopo do produto a todas as partes interessadas no projeto e descreve os principais objetivos do projeto. Além disso, permite que a equipe do projeto realize um planejamento mais detalhado, orientando a execução do projeto, fornecendo linha de base para avaliar solicitações de mudanças ou trabalho adicional (SOMMERVILLE, 2011).

O A&F Health Management é um aplicativo mobile voltado para pessoas idosas, seus cuidadores e/ou familiares. Sua proposta central é organizar e simplificar o gerenciamento de medicamentos e agendamentos de consulta médica. Muitas pessoas idosas precisam lidar com um volume alto de informações críticas, como horários de remédios, consultas marcadas, exames pendentes e restrições médicas. Quando essas informações ficam dispersas em papéis, mensagens, lembretes informais ou na memória, pode aumentar o risco de esquecimentos, atrasos e prejuízos à saúde.

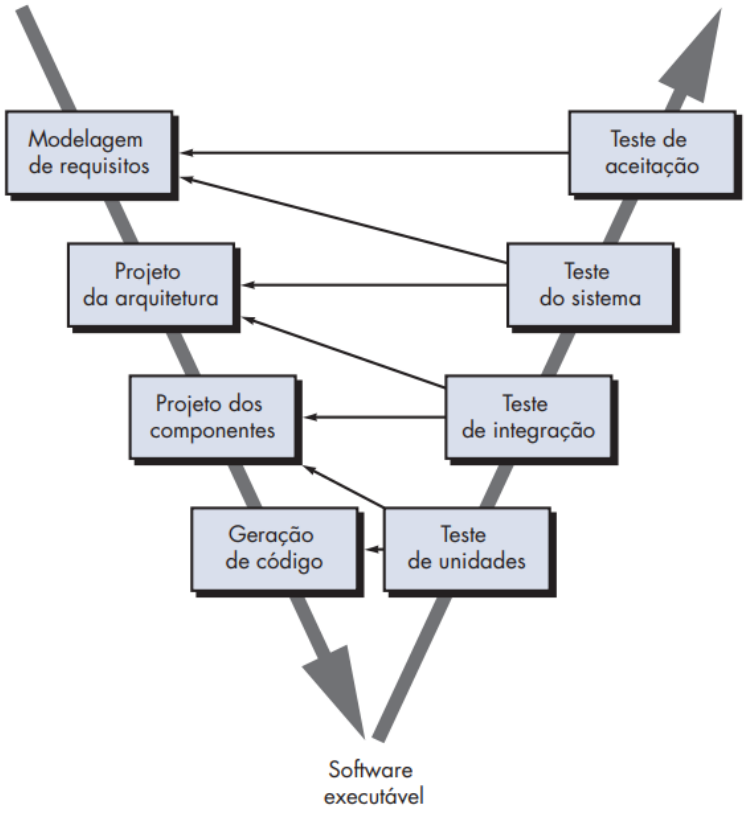
Diante desse cenário, o aplicativo busca reunir essas informações em um ambiente claro, direto e de fácil leitura, ajudando o usuário a lembrar quais medicamentos deve tomar, quando deve tomá-los, quais consultas e exames estão agendados. A proposta não é substituir o acompanhamento médico, mas servir como uma ferramenta de apoio à rotina de cuidado, reduzindo a sobrecarga cognitiva e favorecendo a adesão às recomendações de saúde.

1.2 Objetivo do Projeto

O objetivo principal do projeto é oferecer uma ferramenta *mobile* simples e confiável para o gerenciamento de informações essenciais de saúde. O aplicativo deve permitir que a pessoa idosa, seu cuidador ou seu familiar, mantenha um registro claro das atividades críticas relacionadas à sua saúde, como uso de medicamentos, consultas, exames e restrições médicas.

1.3 Metodologia de desenvolvimento

O projeto foi elaborado como um protótipo de uma pesquisa-ação, quando se elabora uma aplicação prática e colaborativa para a solução de um problema central identificado (APPOLINÁRIO, 2007). O protótipo foi elaborado utilizando como principal abordagem estrutural o modelo de desenvolvimento de *software* em V, no qual cada etapa de desenvolvimento é efetuada levando em consideração os testes de verificação e validação necessários (PRESSMAN, 2011).



Fonte: PRESSMAN, 2011.

A modelagem de requisitos foi construída com base no modelo MoSCoW, classificando os requisitos em essenciais, importantes, desejáveis e não implementados (PAGE-JONES, 2001). A partir disso foram definidos, ao longo do projeto, os requisitos funcionais e não funcionais.

2 MATRIZ DE RASTREABILIDADE DE REQUISITOS DE SISTEMA

Conforme o crescimento do projeto, é necessário que se tenha o controle do fluxo de informações e necessidades em ordem prioritária do projeto (PRESSMAN, 2011). Para isso mapeamos conforme a seguir os requisitos funcionais e não funcionais do projeto.

2.1 Requisitos Funcionais

Nossos requisitos funcionais foram determinados pelo modelo MoSCoW conforme a prioridade do que era considerado essencial para o aplicativo. Segue a tabela completa de requisitos funcionais abaixo.

REQUISITOS FUNCIONAIS (RF)		
ID	**Nome do RF**	**Descrição**
RF01	Entrada de usuário	A permissão de um cadastro e login de usuários do aplicativo
RF02	Divisão de perfis	Cada usuário pode criar múltiplos perfis para que o gerenciamento possa ocorrer levando em consideração diferentes conjuntos de dados.
RF03	Gerenciamento de medicamentos	Agendamentos e notificações referentes a medicamentos que devem ser tomados com regras aplicadas às diferentes cores de tarja.
RF04	Gerenciamento de consultas	Agendamentos e notificações referentes a consultas médicas a serem utilizadas pelos representantes dos perfis incluindo dados obrigatórios e opcionais sobre o hospital onde a consulta será realizada e o médico responsável.
RF05	Registro de restrições médicas	Cada perfil pode ter uma lista de restrições médicas atreladas, de forma a mapear alergias ou outros impedimentos para o uso de determinados medicamentos.

2.2. Requisitos Não Funcionais

Segundo Pressman (2016), requisitos não-funcionais são os requisitos relacionados ao uso da aplicação em termos de desempenho, usabilidade, confiabilidade, segurança, disponibilidade, manutenção e tecnologias envolvidas. Estes requisitos dizem respeito a como as funcionalidades serão entregues ao usuário do software. No caso do A&F Health Management, todos os requisitos não funcionais foram definidos com base na legislação brasileira sobre tratamento de dados sensíveis, atividades exclusivas para portadores de CRM,

normas da Anvisa e recomendações de acessibilidade digital do NIC.br, CGI.br e da W3C.

REQUISITOS NÃO-FUNCIONAIS (RNF)			
RNF	**Característica**	**Subcaracterística**	**Descrição**
1	Promoção à saúde	Alinhamento com a Anvisa	A Agência Nacional de Vigilância Sanitária estabelece normas e recursos digitais que facilitam a compreensão de quais atributos um medicamento pode/deve ter atrelado. Documentos oficiais das normas foram incluídos em todos os prompts de LLM que ofereceram assistência na modelagem de classes.
		Alinhamento com o CRM	O Conselho Federal de Medicina estabelece quais atividades são exclusivas para portadores de CRM, delimitando, legalmente, o escopo de atividades que o aplicativo pode exercer e quais deve deixar às mão de profissionais da medicina.
2	Acessibilidade	Fácil leitura	Fontes legíveis e em larga escala e alto contraste com as cores de fundo para evitar a vista forçada.
		Troca de telas mínima	Dinâmica informacional com o menor número de etapas possível mantendo a clareza das funções.
		Componentes legíveis no modo acessibilidade	Todos os componentes visuais do aplicativo devem ter textos atribuídos ao sistema de voz dos smartphones em modo acessibilidade.
		Alinhamento com W3C	Documentação de diretrizes de acessibilidade da W3C incluída em todos os prompts de LLM que ofereceram assistência na programação de interface do aplicativo.
		Alinhamento com NIC.br e CGI.br	Cartilhas de acessibilidade do NIC.br e CGI.br incluídas em todos os prompts de LLM que ofereceram assistência na programação de interface do aplicativo.
		Alinhamento com a lei brasileira de inclusão das pessoas com deficiência	A documentação oficial do estatuto da pessoa com deficiência foi referência em todos os prompts de LLM durante a programação do Front-End e determinação de requisitos não funcionais.
3	Segurança	Filtragem de atributos com DTO	Classes com dados sensíveis se comportam na API através de DTOs com grupos de atributos filtrados.
		Autenticação JWT	Cada usuário cadastrado possui um token de autenticação.
		Alinhamento com LGPD	A documentação oficial da Lei Geral de Proteção dos Dados foi referência em todos os prompts de LLM durante a programação do back-end.

3 ESTRATÉGIA DE TESTE

A estratégia de teste define quais técnicas, níveis e tipos de testes serão utilizados no projeto (BASTOS et al., 2007). Se tratando de um projeto universitário sem a busca de aprovação do Comitê de Ética em Pesquisa, não foram executadas etapas de validação com usuários reais. Portanto, as estratégias de teste se limitam a verificar requisitos funcionais e aspectos de promoção de requisitos não funcionais de acessibilidade e segurança.

ESTRATÉGIA DE TESTE				
NÍVEIS DE TESTE				
ID	**Nome do Teste**	**Técnicas**	**Descrição**	**Requisitos de Sistemas**
01	Teste Unitário	Testes de caixa branca automatizados com JUnit	Casos de teste documentados transformados em testes automatizados em Java usando a ferramenta JUnit aplicados no Back-End do projeto	RF01, RF02, RF03, RF04, RF05 e RNF3
02	Teste de Integração	Testes de caixa preta realizados manualmente em Smartphone	Fluxo de telas, funcionamento dos componentes e relação das ações realizadas no Smartphone com influência no estado do banco de dados	RF01, RF02, RF03, RF04, RF05, RNF2 e RNF3
TIPOS DE TESTES				
ID	**Tipo do Teste**	**Técnicas** **Descrição**		**Requisitos de Sistemas**
01	Desempenho	Testes desempenho por latência simulando o cadastro de múltiplos usuários simultâneos através do JMeter.		RF01 e RNF3
02	Manutenibilidade	Linting de Back-End Java usando o Checkstyle		N/A
03	Acessibilidade	Diagnóstico de acessibilidade do Front-End usando LightHouse e Wave		RNF2

4 AMBIENTE DE TESTE

O ambiente de testes mostra as condições em que serão executados os testes. Um ambiente de testes isolado busca garantir a descoberta de erros reais no sistema

que está sendo desenvolvido, permitindo que a equipe produza novos códigos sem gerar prejuízo de produção (BASTOS et al., 2007). A equipe deste projeto trabalhou, maioritariamente, à distância, então o acompanhamento de versões do projeto em um ambiente acessível por via de navegação na Internet foi crucial.

Todas as etapas planejadas foram catalogadas, registradas e adequadas a documentos específicos através da ferramenta Trello. Para o desenvolvimento e controle de versões, a equipe gerenciou um repositório no GitHub, criando *branches* sempre que qualquer mudança estrutural na estrutura do projeto fosse necessária. Todos os recursos de ambientes de teste deste projeto consistiram nas máquinas pessoais, computadores e *smartphones*, dos integrantes, que foram usadas para todas as etapas de planejamento, desenvolvimento e verificação.

AMBIENTE DE TESTE	
Recurso	**Descrição**
ANDRÉ COUTINHO	
Computador Pessoal	Processador: Intel(R) Pentium(R) Gold G6400 CPU @ 4.00GHz (4.00 GHz)
	RAM: 16,0 GB
	Placa gráfica: NVIDIA GeForce RTX 2060 (6 GB)
	Armazenamento: 1,95 TB
Mobile	Modelo: POCO X5 5G
	Armazenamento: 128 GB
	Versão OS POCO: 1.0.24.UMPMIXM
	Versão do Android: 14.UKQ1.231003.002
Servidor de banco de dados	H2 Database
Criação de relatórios	Word
	GitHub
Browser	Opera GX - Core: 138.0.6044.65 (PC)
	Google Chrome - Versão 157.0.8103.46 (Mobile)
SO	Windows 11 Home | Versão 25H2
IDE	IntelliJ IDEA 2025
Ecossistema (back-end)	Spring Boot
Testes de desempenho	Apache JMeter
ANDRÉ RODRIGUES	
Computador Pessoal	Processador: 13th Gen Intel(R) Core(TM) i5-13400F (2.50 GHz)
	RAM: 32,0 GB
	Placa gráfica: NVIDIA GeForce RTX 4060 (8 GB)
	Armazenamento: 932 GB

Mobile	Modelo: Galaxy S21 FE 5G | SM-G990E/DS
	Armazenamento: 128 GB
	Versão do Android: 16
Servidor de banco de dados	H2 Database
Visualização do banco de dados	Swagger
Criação de relatórios	GitHub
Browser	Google Chrome - Versão 157.0.8103.39
SO	Windows 11 Pro | Versão 26H2
IDE	IntelliJ IDEA 2025
Ecossistema (back-end)	Spring Boot
Testes de caixa preta (requisições HTTP)	Postman
Testes de caixa preta (Interface Mobile)	Expo Client Version: 58.0.5, Support SDKs: 58
	FERNANDA SOUZA
Computador Pessoal	Processador: 12th Gen Intel(R) Core(TM) i7-1244U (1.70 GHz)
	RAM: 16 GB
	Placa gráfica: Intel(R) Iris(R) Xe Graphics
	Armazenamento: 477 GB
Mobile	Modelo: iPhone 12 | MGJ53BR/A
	Armazenamento: 64 GB
	Versão iOS: 27.0.1
	Número de Série: DV6JL0BM0F0N
Servidor de banco de dados	H2 Database
Criação de relatórios	GitHub
Browser	Google Chrome - Versão 157.0.8103.38
	Safari 27.0.1 (Mobile)
SO	Windows 11 Home Single Language | Versão 25H2
IDE	Visual Studio Code
Framework (front-end)	React Native
Testes de caixa preta (interface mobile)	Expo Client Version: 58.0.5, Supported SDK: 58.0.0

5 MATRIZ DE RESPONSABILIDADE

A matriz de responsabilidade determina a quais integrantes da equipe cabe responsabilidade por cada fase de testes do projeto (BASTOS et al., 2007). Neste projeto, que conta com uma equipe pequena, a responsabilidade do planejamento e regras de condução de testes foi centralizada para uma pessoa. No entanto, todos os

desenvolvedores participaram ativamente da verificação de cada programa atendendo a requisitos mínimos de testes de caixa preta.

MATRIZ DE RESPONSABILIDADE				
Fase do Projeto	**Atribuição Técnica**	**Responsabilidade**	**Nome**	**Descrição da Atividade**
Planejamento	Líder de Projeto de Teste	Técnico responsável pela liderança de um projeto de teste específico, geralmente relacionado a um sistema de desenvolvimento, seja um projeto novo ou uma manutenção.	André Coutinho	Documentação detalhada, pesquisa e organização de etapas do projeto. Toda a pesquisa tomou como base as normas do CFM e da Anvisa aliadas a diretrizes recomendadas pelo CGI.br, pela W3C e leis brasileiras de inclusão e proteção de dados pessoais. A documentação e definição de etapas foi comunicada frequentemente via Trello.
	Arquiteto de teste	É o técnico responsável pela montagem da infraestrutura de teste; monta o ambiente de teste, escolhe as ferramentas de teste e capacita a equipe para executar seu trabalho nesse ambiente de teste.	André Coutinho, André Rodrigues e Fernanda Souza	Montagem de planilhas de casos de teste e de requisitos funcionais e não funcionais, determinação de ferramentas a serem utilizadas em cada teste. Os testes unitários de caixa branca (JUnit) foram feitos para testar recursos funcionais e regras de segurança e atribuindo linting (Checkstyle) para garantia de boas práticas de escrita de código. Para o back-end

				exclusivamente foram feitos testes de desempenho para o cadastro de múltiplos usuários simultâneos (JMeter). No back-end os testes de caixa preta simularam requisições HTTP (Postman) com visualização padrão de bancos de dados (Swagger). No front-end foram feitos testes de caixa preta logo após o desenvolvimento de cada script em interface mobile (Expo). Com a interface do aplicativo adaptada para web, foram elaborados dois diagnósticos de acessibilidade automáticos (Wave e Lighthouse).
Especificação	Analista de Teste	É o técnico responsável pela modelagem e elaboração dos casos de teste e pelos scripts de teste. Em alguns casos, os scripts de teste podem ser elaborados pelos testadores.	André Coutinho	Os scripts de testes unitários automatizados cobriram quatro classes Java atribuindo funções de suas classes de serviço e integração com outras classes.
Execução	Testadores	Aplicação prática dos testes planejados	André Coutinho, André	O integrante André Coutinho se responsabilizou pelos

			Rodrigues e Fernanda Souza	testes unitários de caixa branca no back-end, linting das classes em Java, testes de desempenho e diagnósticos de acessibilidade. O integrante André Rodrigues executou todos os testes de caixa preta do back-end, verificando respostas de requisições HTTP e gerou relatórios de testes de caixa preta do front-end verificando a consistência dos componentes e funções de integração em smartphone Android. A integrante Fernanda Souza realizou os primeiros testes de caixa preta do front-end verificando a dinâmica de botões e transição entre telas em smartphone iOS.

6 RELAÇÃO E DESCRIÇÃO DOS CASOS DE TESTES

Em engenharia de software, o caso de teste é um conjunto de condições usadas para teste de software. Neste plano de teste consta a relação de todos os cenários e seus respectivos casos de testes a serem especificados pelos analistas de teste.

CASOS DE TESTES			
ID	Cenário de Teste	Caso de Teste	Descrição

1	CT_USUÁRIO	CT01_Novos_Usuários	Verificar se novos usuários com dados completos estarão cadastrados no banco.
		CT02_Cadastro_de_Usuário_já_Cadastrado	Verificar se o novo cadastro de um usuário já presente no banco de dados retorna exceção.
		CT03_Cadastro_com_email_invalido	Verificar se o sistema permite a inserção de texto sem '@' como campo de email.
		CT04_Login_de_usuários_válidos	Verificar o acesso do sistema através de email e senha registrados no banco.
		CT05_Login_com_email_inválido	Verificar se a tentativa de acesso com e-mail não cadastrado no banco retorna exceção.
		CT06_Login_com_senha_inválida	Verificar se a tentativa de acesso com e-mail cadastrado, mas com senha inválida, retorna exceção.
		CT07_Login_com_nome_alterado	Verifica a continuidade de andamento do sistema com nome de cadastro alterado.
		CT08_Login_com_email_alterado	Verifica a continuidade de andamento do sistema com email de cadastro alterado.
		CT09_Login_com_senha_alterada	Verifica a continuidade de andamento do sistema com senha de cadastro alterada.
		CT10_Campo_Nome_não_preenchido	Verifica a obrigatoriedade do campo Nome no cadastro
		CT11_Nome_muito_grande	Verifica o limite de tamanho que um nome cadastrado pode ter
		CT12_Nome_com_caracteres_inválidos	Verifica a regra de que o campo nome seja preenchido somente com letras
		CT13_Campo_Email_não_preenchido	Verifica a obrigatoriedade do campo Email no cadastro
		CT14_Email_muito_grande	Verifica o limite de tamanho que um email cadastrado pode ter
		CT15_Campo_Senha_não_preenchido	Verifica a obrigatoriedade do campo Senha no cadastro
		CT16_Senha_muito_curta	Verifica o tamanho mínimo que uma senha cadastrada pode ter
		CT17_Senha_muito_grande	Verifica o limite de tamanho que uma senha cadastrada pode ter

		CT18_Senha_com_caracteres_inválidos	Verifica se a senha não possui emojis ou espaços.
		CT19_Senha_fraca	Verifica se as regras de complexidade da senha estão sendo respeitadas
		CT20_Login_arquivado	Verifica a impossibilidade de fazer login com uma conta arquivada
		CT20_Remember_obrigatorio_para_login_sem_campos	Verifica a entrada automática de usuário somente quando o atributo Remember está ativo
		CT21_Atualização_Inativa	Verifica se a atualização somente será efetuada em um usuário ativo
		CT22_Atualização_com_email_já_existente	Verifica se a atualização barrará a atualização de uma conta com um email já existente
		CT23_Atualização_com_usuario_inexistente	Verifica se a atualização acontece em um usuário existente
2	CT_PERFIL	CT24_Novos_Perfis	Verificar criação de novos perfis no mesmo usuário.
		CT25_Perfil_em_Usuário_não_Existente	Verificar se a tentativa de atrelação de um perfil a usuário não existente retorna exceção.
		CT26_Perfil_com_Nome_Repetido	Verificar se a criação de um novo perfil tem um nome igual ao de outro perfil no mesmo usuário.
		CT27_Lista_de_Perfis	Verificar se os perfis criados em um mesmo usuário estão atribuídos ao mesmo.
		CT28_Lista_em_Usuário_não_Existente	Verificar se a requisição de uma lista de perfis em usuário não existente retorna exceção.
		CT29_Edição_de_Perfil	Verificar se novos parâmetros de perfil estão atrelados ao perfil editado.
		CT30_Edição_de_Perfil_não_Existente	Verificar se a tentativa de edição de um perfil não existente retorna exceção.
		CT31_Edição_de_Perfil_no_Usuário_Errado	Verificar se a falta de vínculo entre perfil editado e usuário retorna exceção.
		CT32_Campo_Nome_não_preenchido	Verifica a obrigatoriedade do campo Nome no cadastro de perfil

		CT33_Nome_muito_grande	Verifica o limite de tamanho que um nome cadastrado de perfil pode ter
		CT34_Nome_com_caracteres_inválidos	Verifica a regra de que o campo nome de perfil seja preenchido somente com letras
		CT35_Avatar_Invalido	Verifica se o ID do Avatar escolhido existe no sistema
		CT36_Editar_perfil_invalido	Verifica se a validade do perfil permanece na edição
		CT37_Editar_Avatar_Invalido	Verifica se a validade do avatar permanece na edição
		CT38_Editar_perfil_inativo	Verifica se a validação de atividade do perfil permanece na edição
		CT39_Editar_perfil_nome_invalido	Verifica se a validação de validade do nome no perfil permanece na edição
		CT40_Editar_perfil_nome_duplicado	Verifica se a validação de unicidade do nome no perfil permanece na edição
3	CT_RESTRIÇÕES	CT41_Nova_lista_de_restrições	Verifica o sucesso na criação de várias restrições em um perfil com todas as características necessárias.
		CT42_Restrições_em_Perfil_válido	Verifica se as restrições estão sendo criadas em um perfil existente e se estão de fato relacionadas ao mesmo.
		CT43_Restrições_em_Perfil_inválido	Verifica se o sistema rejeita restrições em um perfil com ID inválido ou inexistente.
		CT44_Restrições_sem_tipo	Verifica se o sistema rejeita a criação de uma restrição sem o campo Tipo.
		CT45_Restrições_sem_gravidade	Verifica se o sistema rejeita a criação de uma restrição sem o campo Gravidade.
		CT46_Restrição_Atualizada	Verifica o sucesso na atualização de informações de alguma restrição.
		CT47_Atualização_sem_tipo	Verifica se a regra de Tipo obrigatório permanece na atualização.
		CT48_Atualização_sem_gravidade	Verifica se a regra de Gravidade obrigatória permanece na atualização.
		CT49_Atualização_sem_perfil	Verifica se a validação de perfil existente permanece na atualização.

		CT50_Atualização_sem_restrição	Verifica se o sistema rejeita uma tentativa de atualização em uma restrição inexistente.
		CT51_Restrição_sem_Título	Verifica se o sistema rejeita a criação de uma restrição sem título.
		CT52_Título_inválido	Verifica se o sistema rejeita títulos com caracteres inválidos.
		CT53_Título_pequeno	Verifica se o sistema rejeita títulos com menos dígitos do que o requerido.
		CT54_Título_grande	Verifica se o sistema rejeita títulos com mais dígitos do que o permitido.
		CT55_Anotação_inexistente	Verifica se o sistema aceita uma restrição sem anotações.
		CT56_Anotação_grande	Verifica se o sistema rejeita anotações com mais dígitos do que o permitido.
4	CT_AGENDAMENTOS	CT58_Agendamento_atrelado_ao_perfil	Verifica se o agendamento criado está associado ao perfil ao qual está associado.
		CT59_Agendamento_com_perfil_inválido	Verifica se o sistema rejeita um agendamento feito com um perfil de ID inválido ou inexistente.
		CT60_Agendamento_realizado	Verifica o sucesso na criação de um agendamento de medicamento e um de consulta com todos os campos corretamente preenchidos.
		CT61_Agendamento_com_data_inválida	Verifica se o sistema rejeita um agendamento feito em uma data anterior a atual.
		CT62_Agendamento_sem_tipo	Verifica se o sistema rejeita a tentativa de agendamento sem declaração de tipo.
		CT63_Agendamento_sem_título	Verifica se o sistema rejeita a tentativa de agendamento sem título.
		CT64_Agendamento_sem_data_de_início	Verifica se o sistema rejeita a tentativa de agendamento sem uma data de início.
		CT65_Agendamento_sem_intervalo	Verifica se o sistema rejeita a tentativa de agendamento sem o atributo de intervalo.
		CT66_Agendamento_com_anotação_muito_longa	Verifica se o sistema rejeita um atributo de anotações com tamanho maior que 1000.

		CT67_Agendamento_sem_confirmação	Verifica se o sistema rejeita a tentativa de agendamento sem estar associado à confirmação.
		CT68_Atualização_de_agendamento	Verifica o sucesso na atualização de informações de um agendamento existente.
		CT69_Atualização_em_agendamento_inválido	Verifica se o sistema rejeita a tentativa de atualização em um agendamento com ID inválido ou inexistente.
		CT70_Atualização_em_agendamento_inativo	Verifica se o sistema rejeita a tentativa de atualização em agendamento existente, porém inativo.
		CT71_Atualização_com_mudança_de_tipo	Verifica se o sistema rejeita a tentativa de atualização no campo Tipo de agendamento.
		CT72_Atualização_de_medicamento_não_agendado	Verifica se o sistema rejeita a atualização de um medicamento sem dados relacionados.
		CT73_Atualização_de_consulta_não_agendada	Verifica se o sistema rejeita a atualização de uma consulta sem dados relacionados.
		CT73_Remoção_de_agendamento	Verifica o sucesso na remoção de um agendamento usando o delete.
		CT74_Remoção_de_agendamento_inválido	Verifica se o sistema rejeita a remoção de um agendamento com ID inválido ou inexistente.
		CT75_Remoção_de_agendamento_inativo	Verifica se o sistema rejeita a remoção de um agendamento existente, porém inativo.
		CT76_Medicamento_sem_nome	Verifica se o sistema rejeita a criação de um medicamento sem nome.
		CT77_Atualização_de_medicamento_sem_nome	Verifica se a regra de nome do medicamento obrigatório permanece na atualização.
		CT78_Medicamento_sem_unidade_de_medida	Verifica se o sistema rejeita a criação de um medicamento sem declaração da unidade de dosagem.
		CT79_Atualização_de_medicamento_sem_unidade_de_medida	Verifica se a regra de unidade de medida do medicamento obrigatório permanece na atualização.
		CT80_Consulta_sem_nome_de_médico	Verifica se o sistema aceita a criação de uma consulta sem o nome do médico.

		CT81_Atualização_de_consulta_sem_nome_de_médico	Verifica se a regra de não obrigatoriedade de nome do médico permanece na atualização.
		CT82_Consulta_sem_localização	Verifica se o sistema aceita a criação de uma consulta sem localização.
		CT83_Atualização_de_consulta_sem_localização	Verifica se a regra de não obrigatoriedade de localização permanece na atualização.
		CT84_Consulta_sem_título	Verifica se o sistema rejeita a criação de uma consulta sem título
		CT85_Atualização_de_consulta_sem_título	Verifica se a regra de obrigatoriedade de título da consulta permanece na atualização.
		CT86_Consulta_com_título_de_tamanho_inválido	Verifica se o sistema rejeita títulos de consulta fora dos valores limite de tamanho.
		CT87_Atualização_de_consulta_com_título_inválido	Verifica se a regra de tamanho do título da consulta permanece na atualização.
		CT88_Consulta_com_título_inválido	Verifica se o sistema rejeita títulos de consulta com caracteres especiais e números.
		CT89_Atualização_de_consulta_com_título_inválido	Verifica se a regra de validade do título permanece na atualização.
		CT90_Texto_Opcional_inexistente	Verifica se o sistema aceita um agendamento sem texto opcional.
		CT91_Atualização_sem_texto_opcional	Verifica se a não obrigatoriedade do texto opcional permanece na atualização.
		CT92_Texto_opcional_muito_longo	Verifica se o sistema rejeita textos opcionais fora dos valores limite de tamanho.
		CT93_Atualização_de_texto_opcional_muito_longo	Verifica se a regra de tamanho do textos opcionais permanece na atualização.

6.1 Descrição dos principais cenários de testes

Os principais e mais abordados cenários de teste estão relacionados à classe “Schedule”, que compõe todas as funções relacionadas ao agendamento de consultas

médicas e lembretes de medicamentos. Para o cadastro de um medicamento, por exemplo, é obrigatório o preenchimento do campo “nome”. Se for tentado um cadastro de medicamento sem nome, o sistema deve retornar uma exceção específica informando o cliente da requisição que este é o campo que está faltando. O estado do medicamento só passará a ser considerado um evento válido, dentro dessa e outras condições

Outro exemplo é o campo “título” da consulta. Não só é obrigatório como tem um mínimo de 3 caracteres e um máximo de 100 caracteres. Nesse caso de teste se utiliza a partição de equivalência por valor limite. O mesmo se aplica a campos opcionais como o “texto opcional”, que não precisa necessariamente ser incluído, mas se for, deve ter um máximo de 1000 caracteres.

7 RISCOS DO PROJETO DE TESTE DE SOFTWARE

Este capítulo contém um sumário dos riscos chaves do projeto de teste, que se acontecerem, poderão afetar os resultados esperados e o projeto de software (RIOS, 2005). Neste projeto todos os testes unitários foram feitos em *branches* exclusivas no repositório. Mesmo assim os riscos do projeto foram documentados como uma forma de prever inconveniências no desenvolvimento, principalmente considerando que cada integrante do projeto trabalhou em sua própria máquina e o acompanhamento de cada etapa foi feito à distância.

RISCOS DO PROJETO DE TESTE				
ID	**Risco**	**Descrição**	**Grau do Risco**	**Resposta**
01	Hardware	Risco do equipamento (hardware) utilizado para execução da homologação do teste falhar.	Baixo impacto; Média probabilidade	Aceitar – reiniciar o equipamento ou tentar com outra máquina
02	Incompatibilidade de sistemas mobile	Risco dos programas criados em React Native e testados no sistema	Alto impacto; Alta probabilidade	Evitar – garantir responsividade na programação

		iOS não funcionarem da mesma maneira no sistema Android.		da interface e construção dos componentes e evitar usar componentes exclusivos de um dos dois sistemas. Sempre que for necessário usar um componente exclusivo iOS ou Android, sempre se certificar de que um equivalente está sendo criado para o outro sistema.
03	Conflito de dependências	Risco que branches do repositório façam alterações significativas na lista de dependências do projeto Spring (pom.xml).	Médio impacto; Alta probabilidade	Mitigar – configurar o repositório para sempre pedir revisões dos outros desenvolvedores antes de aceitar qualquer pull request; manter branches de testes exclusivas para si, nunca totalmente integradas na main com merge, a menos que o projeto seja finalizado e

				as diferenças de commit sejam aplicadas manualmente.
04	Barreiras de segurança	Risco em que as regras de segurança aplicadas ao back-end impeçam o andamento de testes de desempenho.	Alto impacto; Alta probabilidade	Evitar – Fazer modificações no script back-end diminuindo as barreiras de segurança na branch de testes. Deve-se ter em mente que tais scripts alterados nunca deverão ser "commitados" para a branch main.

8 CRITÉRIOS DE CONCLUSÃO

Tratando-se um projeto universitário, a equipe reconhece que o projeto não estará completo em um futuro próximo. Os critérios de conclusão dos testes levam em consideração o tempo determinado para entregar cada etapa do projeto, incluindo testes específicos. Foi estabelecido que os principais testes que devem ser concluídos são aqueles vinculados aos critérios principais de qualidade determinados: promoção à saúde, acessibilidade e segurança.

Critério	Descrição
Cobertura	▪ Todos os requisitos definidos no escopo deste projeto deverão ser testados. ▪ Testar fluxos básico, alternativo e de exceção de cada requisito funcional. ▪ Todas as regras de negócio serão testadas. ▪ Todas as classes Java funcionais no serviço da API deverão ter sua própria classe de testes unitários.

Critério	Descrição
	▪ Todas as telas programadas deverão ter sua aparição confirmada nos testes. ▪ Todas as mensagens de erro vinculadas a problemas com o código back-end devem ser previstas e manualmente descritas. ▪ Todos os componentes e scripts vinculados aos requisitos não funcionais devem ser minimamente testados.
Quando parar de testar	▪ Quando nenhuma falha for encontrada após executar todos os casos de teste pelo menos uma vez. ▪ Quando for encontrada alguma falha que inviabilize a continuidade dos testes. ▪ Quando atingir no mínimo os seguintes critérios: o Casos de Teste ▪ 100% dos casos de teste executados o Diagnósticos ▪ Pontuação de acessibilidade maior ou igual a 95 no Ligthouse. ▪ Pontuação de acessibilidade maior ou igual a 7.0 no Wave[1]. o Linting ▪ 0 violações detectadas pelo Checkstyle.
Em que momento reportar as falhas	▪ Reportar todas as falhas por Whatsapp, GitHub e/ou Trello imediatamente após encontrá-las.
Teste de regressão	▪ Feito por reiniciação de hardware somente quando suspeita de falha de compilação das IDEs ou navegadores.

9 CRONOGRAMA DO PROJETO DE TESTE DE SOFTWARE

O cronograma serve para que a equipe siga um rumo de atividades realista com a expectativa de entrega (PRESSMAN, 2016). Para este projeto foi considerado o período de um semestre para a criação e verificação de um protótipo do aplicativo A&F Health Management.

[1] O Wave adiciona erros conforme a ausência de estrutura de cabeçalho e falta de textos alternativos embutidos, o que é uma garantia para um sistema mobile baseado em React Native que não possui esses atributos. Por esse único motivo a expectativa de pontuação no Wave é naturalmente mediana.

Mês.-Ano	Atividade(s)	Descrição
ago.-26	**Primeira versão da documentação do projeto**	Documento contendo a divisão de tarefas dos integrantes, escopo e objetivos do projeto.
set.-26	**Estruturação de classes no back-end**	Primeiras classes em Java programadas, estruturadas e funcionais com requisições HTTP básicas (CRUD via REST)
out.-26	**Segunda versão da documentação do projeto**	Documento atualizado com as atividades envolvendo back-registradas.
	Testes de desempenho	Sequência de testes realizados via JMeter para analisar o desempenho da API com cadastros de múltiplos usuários simultâneos.
nov.-26	**Back-end finalizado**	Todas as classes de entidades, repositórios, serviços, controladores, enumeradores e DTOs finalizados considerando o escopo inicial do projeto.
	Testes unitários back-end	Scripts de teste em Java (JUnit) para verificação de funções e regras de segurança.
	Linting de classes Java	Refinamentos da escrita de código influenciadas por um diagnóstico gerado com Checkstyle.
	Estruturação de componentes React Native	Primeiros componentes React montados no framework React Native para uma visualização inicial e minimamente interativa do aplicativo.
	Front-end finalizado	Todos os componentes React Native finalizados considerando o escopo inicial do projeto.
	Diagnósticos de Acessibilidade	Análises automáticas de recursos front-end de acessibilidade feitos com Lighthouse e Wave.
	Integração de front-end e back-end sólida	Requisitos funcionais do projeto em pleno funcionamento envolvendo ações no front-end que desencadeiam atividades no back-end.

	Terceira versão da documentação do projeto	Documento atualizado com as atividades envolvendo back-end, front-end e integração finalizadas.
	Documentação de plano de testes finalizada	Documento contendo todos os detalhes do planejamento de testes.
	Relatório de sumário dos testes finalizado	Documento contendo os resultados e evidências de todos os testes planejados e documentados no documento de plano de testes.
	Apresentação de QA do projeto	Apresentação formal, visual e oral dos integrantes da equipe com os recursos e evidências de QA do projeto.
	Apresentação do projeto para stakeholders (professores)	Apresentação formal, visual e oral dos integrantes da equipe com o projeto funcionando e com todos os seus recursos explicados.

REFERÊNCIAS

AGÊNCIA NACIONAL DE VIGILÂNCIA SANITÁRIA (Anvisa). Instruções de Integração API SNCR v1.0: instruções de apoio ao processo de integração. **Anvisa**, Brasília, DF, 2026.

AGÊNCIA NACIONAL DE VIGILÂNCIA SANITÁRIA (Anvisa). SNCR Manual da API. 2. ed. **Anvisa**, Brasília, DF, 2026.

AGÊNCIA NACIONAL DE VIGILÂNCIA SANITÁRIA (Anvisa). SNCR Manual do Usuário. 6. ed. **Anvisa**, Brasília, DF, 2026.

APPOLINÁRIO, Fábio. **Dicionário de metodologia científica: um guia para a produção do conhecimento científico**. São Paulo: Editora Atlas. 2007.

BASTOS, Anderson et al. **Base de conhecimento em teste de software**. São Paulo: Editora Martins Fontes, v. 30, 2007.

BRASIL. Lei nº 13.146, de 6 de julho de 2015. **Institui a Lei Brasileira de Inclusão da Pessoa com Deficiência (Estatuto da Pessoa com Deficiência)**. Diário Oficial da União: seção 1, Brasília, DF, 7 jul. 2015.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. **Dispõe sobre o tratamento de dados pessoais, inclusive nos meios digitais, por pessoa natural ou por pessoa jurídica de direito público ou privado, com o objetivo de proteger os direitos fundamentais de liberdade e de privacidade e o livre desenvolvimento da personalidade da pessoa natural**. Diário Oficial da União: seção 1, Brasília, DF, 15 ago. 2018.

BRASIL. Ministério da Gestão e da Inovação em Serviços Públicos. Guia de Boas Práticas para Acessibilidade Digital. **Programa de Cooperação entre Reino Unido e Brasil em Acesso Digital**. SGD/MGI, Brasília, DF, 2023.

BRASIL. Ministério da Saúde. Portaria SVS/MS nº 344, de 12 de maio de 1998. **Aprova o Regulamento Técnico sobre substâncias e medicamentos sujeitos a controle especial**. Diário Oficial da União: seção 1, Brasília, DF, 1998.

BRASIL. Ministério do Planejamento, Orçamento e Gestão. **eMAG Modelo de Acessibilidade em Governo Eletrônico**. Versão 3.1. MP, SLTI, Brasília, DF, 2014.

COMITÊ GESTOR DA INTERNET NO BRASIL (CGI.br). Cartilha de Acessibilidade na Web do W3C Brasil – Fascículo II: Benefícios, Legislação e Diretrizes de Acessibilidade na Web. **CGI.br**, São Paulo, 2015.

COMITÊ GESTOR DA INTERNET NO BRASIL (CGI.br). Cartilha de Acessibilidade na Web do W3C Capítulo São Paulo – Fascículo VI: Avaliando acessibilidade e resultados. **CGI.br**, São Paulo, 2023.

CONSELHO FEDERAL DE MEDICINA (Brasil). Resolução CFM nº 2.299, de 30 de setembro de 2021. Autoriza a utilização de Tecnologias Digitais da Informação e Comunicação (TDICs) para a emissão dos documentos médicos. **CFM**, Brasília, DF, 2021.

CONSELHO FEDERAL DE MEDICINA (Brasil). Resolução CFM nº 2.309, de 22 de março de 2022. Estabelece regramento para publicização e compartilhamento de dados de médicos inscritos à luz da LGPD, do interesse público e das atribuições legais conferidas ao Conselho Médico. **CFM**, Brasília, DF, 2022.

DELAMARO, Márcio Eduardo; MALDONADO, José Carlos; JINO, Mario. **Introdução ao Teste de Software**. São Paulo: Elsevier, 2007.

MOLINARI, Leonardo. **Inovação e Automação de Testes de Software**. Rio de Janeiro: Érica, 2010.

PAGE-JONES, Meilir. **Fundamentos do desenho orientado a objeto com UML**. São Paulo: Pearson, 2001.

PRESSMAN, Roger S; MAXIM, Bruce R. **Engenharia de Software: Uma Abordagem Profissional**. 8ª ed. Editora McGraw-Hill, 2016.

RIOS, Emerson. **Análise de riscos: em projetos de teste de software**. Rio de Janeiro: Alta Books, 2005.

RIOS, Emerson; MOREIRA FILHO, Trayahú. **Teste de Software**. Rio de Janeiro: Alta Books, 2006.

SOMMERVILLE, Ian. **Engenharia de software**. 9. ed. São Paulo: Pearson, 2011.

WORLD WIDE WEB CONSORTIUM (W3C). Diretrizes de Acessibilidade para Conteúdo Web (WCAG) 2.0. Tradução autorizada para o português do Brasil pelo W3C Brasil. **W3C Brasil**, São Paulo, 2014.

WORLD WIDE WEB CONSORTIUM (W3C). Web Content Accessibility Guidelines (WCAG) 2.0. W3C Recommendation 11 December 2008. **W3C**, [S.l.], 2008.